ESSENTIALS OF DATA SCIENCE
Assignment No. 2: NumPy and Pandas

Name: Ashutosh Sharma
PRN: 202501070190
Division: ET2

Dataset: IMDB Movie Reviews Dataset

The IMDB dataset contains movie reviews along with their sentiment labels (positive or negative). It is widely used for sentiment analysis and text classification tasks. The dataset includes textual review data, making it suitable for both statistical and text-based analysis using NumPy and Pandas.

Problem Statements

1. Load the IMDB dataset using Pandas and display the first 10 rows.

```
import pandas as pd
import numpy as np
from collections import Counter

# Load dataset
df = pd.read_csv("D:\code\ssignments\IMDB Dataset.csv")

# 1. First ten rows
print("\n--- First 10 Rows ---")
print(df.head(10))
```

```
--- First 10 Rows ---
                                              review sentiment
0  One of the other reviewers has mentioned that ...  positive
1  A wonderful little production. <br /><br />The...  positive
2  I thought this was a wonderful way to spend ti...  positive
3  Basically there's a family where a little boy ...  negative
4  Petter Mattei's "Love in the Time of Money" is...  positive
5  Probably my all-time favorite movie, a story o...  positive
6  I sure would like to see a resurrection of a u...  positive
7  This show was an amazing, fresh & innovative i...  negative
8  Encouraged by the positive comments about this...  negative
9  If you like original gut wrenching laughter yo...  positive
```

2. Check for missing or null values in the dataset.

```
# 2. Missing values
print("\n--- Missing Values ---")
print(df.isnull().sum())

```

```
--- Missing Values ---
review       0
sentiment    0
dtype: int64
```

3. Display the data types of all columns.

```
# 3. Data types
print("\n--- Data Types ---")
print(df.dtypes)

```

```
--- Data Types ---
review       str
sentiment    str
dtype: object
```

4. Find the total number of reviews in the dataset.

```
# 4. Total reviews
print("\nTotal reviews:", len(df))

```

```
Total reviews: 50000
```

5. Convert sentiment labels (positive/negative) into numeric format.

```
# 5. Convert sentiment to numeric
df['sentiment_numeric'] = df['sentiment'].map({'positive':1, 'negative':0})

```

```
--- Sentiment Count ---
sentiment
positive    25000
negative    25000
Name: count, dtype: int64
```

6. Create a new column representing the length of each review (in characters).

```
# 6. Review length
df['review_length'] = df['review'].astype(str).apply(len)
```

7. Create a new column representing the word count of each review.

```
# 7. Word count
df['word_count'] = df['review'].astype(str).apply(lambda x: len(x.split()))

```

8. Calculate the average review length.

```
# 8. Mean length
lengths = df['review_length'].values
print("Average length of review",np.mean(lengths))

```

```
Average length of review 1309.43102
```

9. Compute median, and standard deviation of review length using NumPy.

```python
# 9. Median, Standard Deviation of review length
print("\n--- Median,Standard Deviation of review length ---")
print("Median:", np.median(lengths))
print("Std Dev:", np.std(lengths))
```

```
--- Median,Standard Deviation of review length ---
Median: 970.0
Std Dev: 989.7181170827175
```

10.Find the maximum and minimum review length.

```python
# 10. Max & Min length
print("\nMax length:", df['review_length'].max())
print("Min length:", df['review_length'].min())
```

```
Max length: 13704
Min length: 32
```

11.Calculate the variance of review length using NumPy.

```python
# 11. Variance in review length
print("\n--- Variance in review length ---")
print("Variance:", np.var(lengths))
```

```
--- Variance in review length ---
Variance: 979541.9512817597
```

12.Compare average review length for positive and negative sentiments.

```python
# 12. Avg length by sentiment
print("\n--- Avg Length by Sentiment ---")
print(df.groupby('sentiment')['review_length'].mean())
```

```
--- Avg Length by Sentiment ---
sentiment
negative    1294.06436
positive    1324.79768
Name: review_length, dtype: float64
```

13.Count the total number of positive and negative reviews.

```python
# 13. Sentiment counts
print("\n--- Sentiment Count ---")
print(df['sentiment'].value_counts())
```

```
--- Sentiment Count ---
sentiment
positive    25000
negative    25000
Name: count, dtype: int64
```

14.Calculate the percentage distribution of sentiments.

```python
# 14. Percentage distribution
print("\n--- Sentiment % ---")
print(df['sentiment'].value_counts(normalize=True) * 100)

```

```
--- Sentiment % ---
sentiment
positive    50.0
negative    50.0
Name: proportion, dtype: float64
```

15.Filter reviews longer than 500 characters.

```python
# 15. Long reviews (>500 chars)
print("\n--- Long Reviews ---")
print(df[df['review_length'] > 500].head(5))

```

```
--- Long Reviews ---
                                              review sentiment  sentiment_numeric  review_length  word_count
0  One of the other reviewers has mentioned that ...  positive                  1           1761         307
1  A wonderful little production. <br /><br />The...  positive                  1            998         162
2  I thought this was a wonderful way to spend ti...  positive                  1            926         166
3  Basically there's a family where a little boy ...  negative                  0            748         138
4  Petter Mattei's "Love in the Time of Money" is...  positive                  1           1317         230
```

16.Identify reviews with more than 100 words.

```python
# 16. Reviews >100 words
print("\n--- Wordy Reviews ---")
print(df[df['word_count'] > 100].head(5))

```

```
--- Wordy Reviews ---
                                              review sentiment  sentiment_numeric  review_length  word_count
0  One of the other reviewers has mentioned that ...  positive                  1           1761         307
1  A wonderful little production. <br /><br />The...  positive                  1            998         162
2  I thought this was a wonderful way to spend ti...  positive                  1            926         166
3  Basically there's a family where a little boy ...  negative                  0            748         138
4  Petter Mattei's "Love in the Time of Money" is...  positive                  1           1317         230
```

17.Filter all negative reviews.

```python
# 17. Negative reviews
print("\n--- Negative Reviews ---")
print(df[df['sentiment'] == 'negative'].head(5))

```

```
--- Negative Reviews ---
                                               review sentiment  sentiment_numeric  review_length  word_count
3   Basically there's a family where a little boy ...  negative                  0            748         138
7   This show was an amazing, fresh & innovative i...  negative                  0            934         174
8   Encouraged by the positive comments about this...  negative                  0            681         130
10  Phil the Alien is one of those quirky films wh...  negative                  0            578          96
11  I saw this movie when I was about 12 when it c...  negative                  0            937         180
```

18.Identify the most frequent words in positive reviews.

```python
# 18. Frequent words (positive)
positive_words = " ".join(df[df['sentiment']=='positive']['review']).lower().split()
pos_freq = Counter(positive_words)

print("\n--- Top Positive Words ---")
print(pos_freq.most_common(10))

```

```
--- Top Positive Words ---
[('the', 326268), ('and', 171379), ('a', 160990), ('of', 150754), ('to', 129608), ('is', 108994), ('in', 95642), ('i', 66
833), ('it', 64793), ('this', 64167)]
```

19.Identify the most frequent words in negative reviews.

```python
# 19. Frequent words (negative)
negative_words = " ".join(df[df['sentiment']=='negative']['review']).lower().split()
neg_freq = Counter(negative_words)

print("\n--- Top Negative Words ---")
print(neg_freq.most_common(10))

```

```
--- Top Negative Words ---
[('the', 312593), ('a', 155625), ('and', 142258), ('of', 135907), ('to', 134965), ('is', 95882), ('in', 84165), ('i', 747
54), ('this', 74316), ('that', 66232)]
```

20.Create a simple rule-based classifier and evaluate its accuracy.

```python
# 20. Simple classifier
def simple_classifier(text):
    text = text.lower()
    if "good" in text or "excellent" in text:
        return 1
    else:
        return 0

df['predicted'] = df['review'].apply(simple_classifier)

accuracy = (df['predicted'] == df['sentiment_numeric']).mean()
print("\nSimple Model Accuracy:", accuracy)
```

```
Simple Model Accuracy: 0.52356
```

Conclusion

The IMDB dataset was analyzed using NumPy and Pandas to perform data cleaning, statistical analysis, and text processing. The dataset showed a balanced distribution of positive and negative sentiments. Review length analysis and word frequency analysis provided insights into user opinions. A simple rule-based classifier was implemented to classify sentiments, demonstrating basic text classification techniques.